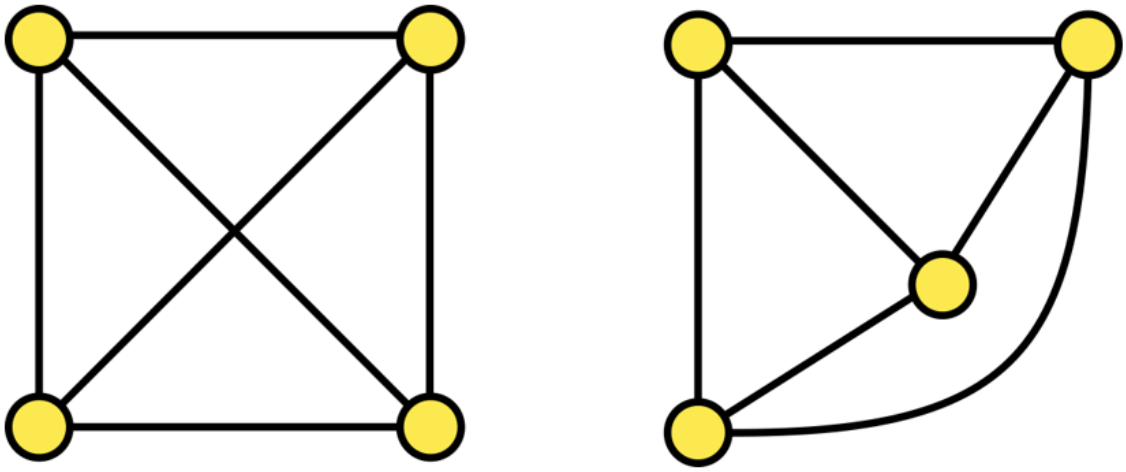


Figure 16.1 Two embeddings for complete graph K_4: the one on the left is not optimal because there is a pair of edges crossing.

Well, it seems there is not much we can do… or is there?

In this chapter, we are going to introduce heuristics to embed graphs with as few intersections as possible, approaching its crossing number: if you refer to figure 16.1, the goal of these heuristics, when applied for instance to graph K_4, is to find the embedding on the right, or an equivalent one with no edges crossing. We'll start with naïve, brute-force-style algorithms, and then refine them to obtain better results (fewer intersections) faster[2]: we'll talk about optimization.

Across the following sections we'll define the category of optimization problems, which has deep implications on our daily lives: network routing, delivery schedule, circuit boards printing, component design – these problems can all be expressed in terms of graphs, a cost function, and an optimization algorithm whose goal is to find the minimum cost configuration. We'll discuss three different optimization techniques to find approximate solutions for them: *random sampling*, *hill climbing*, and *gradient descent*.